

Neural Models Lab

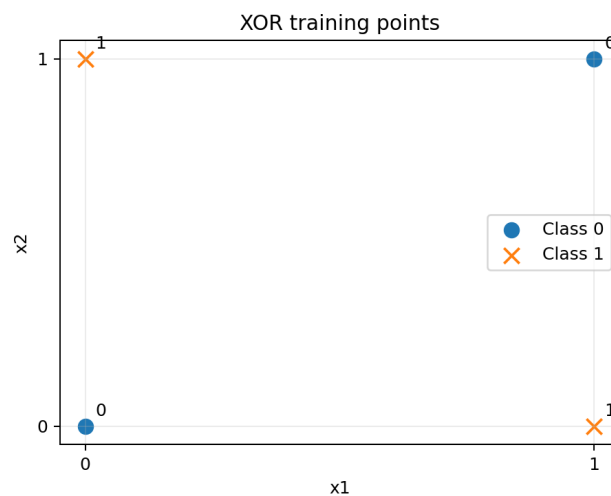
Learning, Depth, Activations, and Output Layers

This report follows the submission requirements in the Neural Models hands-on. The experiments use a small 2-2-1 network for XOR and then extend the same input to a three-class problem.

1. Task 1 - Problem specification

Input space: $X = \{0,1\} \times \{0,1\}$. **Output:** $Y = \{0,1\}$, where 1 means the two sensors disagree.

x1	x2	y	Meaning
0	0	0	both inactive
0	1	1	disagreement
1	0	1	disagreement
1	1	0	both active



A single straight decision boundary cannot separate XOR because the class-1 points are on opposite corners while the class-0 points occupy the other two corners. A single affine transformation followed by a sigmoid is still a linear decision boundary, so it cannot represent XOR exactly.

2. Task 2 - Model design

Architecture: 2 inputs -> 2 hidden units -> 1 output logit. For the main experiment I used tanh in the hidden layer, random initialization, Adam optimization, full-batch training for 3000 steps, and BCEWithLogitsLoss. The output probability is obtained using sigmoid only when reporting predictions.

The hidden nonlinearity is needed because without it, a stack of affine layers reduces to one affine map. That still cannot represent XOR. Sigmoid with binary cross-entropy is appropriate for a yes/no target; using logits with BCEWithLogitsLoss is the PyTorch version I used in the code.

Validation checks: final loss should fall substantially, all four XOR labels should be correct, the first-layer gradient should be non-zero early in training, and changing the hidden activation should give measurable differences in the training result.

3. Task 3 - LLM prompt and code changes

Prompt used: Generate minimal PyTorch code for a 2-2-1 XOR network using the four examples (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Use a nonlinear hidden activation, one output logit, BCEWithLogitsLoss, random initialization, full-batch training for a few thousand CPU steps, and report the initial/final loss, four probabilities, thresholded labels, and one first-layer gradient tensor. Also include a zero-initialization symmetry test and an extension with three output logits using CrossEntropyLoss.

Changes I made after inspecting the generated code: I added a fixed random seed so the runs were repeatable, and I added explicit printing of the first-layer gradient plus the activation-comparison results. I also kept the output as logits and used BCEWithLogitsLoss, which matches the lab's PyTorch recommendation.

Before running it, I checked that the forward pass happens in the model's forward() method, the loss is formed with BCEWithLogitsLoss, backward() is called before optimizer.step(), and the optimizer is the part that updates the parameters.

4. Task 4 - Binary XOR results

Measure	Result
Activation	tanh
Seed	2
Steps	3000
Learning rate	0.05
Initial loss	0.693229
Final loss	0.000090
Predicted probabilities	[0.000070, 0.999901, 0.999876, 0.000068]
Predicted labels	[0, 1, 1, 0]
Correct	4/4
Early grad W1 2	0.005077

The four examples were classified correctly. The loss fell from about 0.6932 to 9.0e-5. The early first-layer gradient was non-zero, and the final probabilities were close to 0 or 1.

5. Backpropagation check

First-layer gradient after the first backward():

```
[ [-0.000575  0.00048 ]
 [ 0.002101  0.004561]]
```

The tensor stored in parameter.grad represents the derivative of the scalar loss with respect to the corresponding first-layer weights, i.e. $dL/dW(1)$. Because the loss uses the mean over the four training examples, this gradient is the average of their individual gradients.

6. Zero-initialisation symmetry experiment

Step	Hidden weight row 1	Hidden weight row 2	Identical?
0	[0.0, 0.0]	[0.0, 0.0]	Yes
1	[0.0, 0.0]	[0.0, 0.0]	Yes
2	[0.0, 0.0]	[0.0, 0.0]	Yes
3	[0.0, 0.0]	[0.0, 0.0]	Yes
4	[0.0, 0.0]	[0.0, 0.0]	Yes
5	[0.0, 0.0]	[0.0, 0.0]	Yes

With every weight and bias initialized to zero, the two hidden units stay identical. In this run the hidden-layer weights remain zero because the output weights also start at zero, so the hidden layer receives zero gradient. This shows the symmetry problem: the two hidden units have no mechanism to start learning different features.

7. Activation experiment

Hidden activation	Final loss	4/4 correct?	Early grad W1 2
sigmoid	0.00025914	Yes	0.005453

tanh	0.00009038	Yes	0.005077
relu	0.00003288	Yes	0.004868

All three activations learned the four XOR labels correctly in this particular run, but they produced different losses and early gradient norms. The scientific reason is that their derivatives are different: sigmoid can have small derivatives when saturated, tanh also changes the gradient scale, and ReLU has derivative 1 when active and 0 when its input is negative. The table is an observation from this experiment, not a claim that one activation is always best.

8. Task 5 - Three-class extension

The new labels are 0 for (0,0), 1 for disagreement, and 2 for (1,1). The hidden layer is still 2 units, but the final linear layer now has shape **(3, 2)**, so each example produces three logits. I used CrossEntropyLoss.

Input	Target	P(class 0)	P(class 1)	P(class 2)	Prediction
[0, 0]	0	0.999973	0.000027	0.000000	0
[0, 1]	1	0.000009	0.999985	0.000006	1
[1, 0]	1	0.000008	0.999985	0.000007	1
[1, 1]	2	0.000000	0.000024	0.999976	2

Initial loss: 1.040851. Final loss: 0.00002044. All four examples were classified correctly. For the first example, the softmax probabilities sum to 0.99999994. Adding 100 to all three logits changes the probability vector only by about $2.42\text{e-}13$, which is just floating-point roundoff. This is why stable softmax implementations can subtract the maximum logit before exponentiating.

For cross-entropy, the gradient of the loss with respect to the logits is $p - y$. Softmax converts the logits into a probability distribution, so the components sum to one.

9. Reflection Questions

1. What did the XOR experiment demonstrate about depth and nonlinearity?

It showed that adding more affine layers is not enough. Without a nonlinear hidden activation, the layers collapse to one affine transformation, which still cannot represent XOR. The useful part of the hidden layer is the nonlinearity.

2. What showed that backpropagation supplied a useful learning signal?

The first-layer gradient was non-zero at the beginning, and the loss then dropped from about 0.6932 to $9.0e-5$ while all four labels became correct. So the gradients were not just mathematically non-zero; they led to parameter updates that improved the predictions.

3. Why did identical/zero initialization prevent distinct hidden features?

The two hidden units started in exactly the same state and therefore produced the same output and received the same gradient. With all weights at zero, the hidden weights stayed identical, so there was no symmetry breaking.

4. How did the hidden activation affect the gradient?

The early gradient norm was slightly different for sigmoid, tanh and ReLU. Scientifically, this comes from their different derivatives. The exact numbers in the table are an engineering observation from this four-point experiment.

5. Why must output layer and loss be selected together?

The output format should match the prediction task. A binary target can use one logit with BCEWithLogitsLoss, while a one-of-three-class task needs three logits with multiclass cross-entropy. Using the wrong pairing would not give the intended probability interpretation or gradient.

6. One example where the LLM helped and one where verification was needed

The LLM made it faster to write the initial PyTorch structure and training loop. I still had to check the architecture, loss, gradients and experiment outputs because a program can run without actually implementing the required experiment correctly.

7. Which tests would remain useful if the model were larger?

I would keep loss/prediction checks, gradient inspection on selected parameters, sanity checks on output probabilities, and controlled ablations. Exhaustive finite-difference gradient checks would become too expensive, so for a larger model I would use them only on small subsets or on a reduced test case.

10. Short personal reflection

I used an LLM as a coding assistant rather than treating the generated code as the answer. The useful part was getting a clean PyTorch starting point quickly. The important part of the lab was then checking whether the implementation matched the model I had specified, especially the loss function, the gradient tensor, the zero-initialisation experiment, and the three-class output. The XOR example made the difference between representation and optimization much easier to see because the dataset is small enough to inspect manually.

11. Files included with this report

neural_models_lab.py - final code for the binary XOR experiment, symmetry test, activation comparison, and three-class extension.

activation_results.csv - compact record of the activation experiment results.